Software Design Document

Face Recognition-Based Smart Attendance Management System

Project ID: 20

Version 1.0

UE24CS341A - Software Engineering, Jackfruit Mini-Project

Team Details

Field	Details
Team Name	SE GROUP
Team Member 1	MITHUN S | SRN: PES2UG24AM208
Team Member 2	BHARATH H G | SRN: PES2UG24AM197
Team Member 3	VARUN C H | SRN: PES2UG24AM180
Team Member 4	YASHWANT ADITYA VARMA | SRN: PES2UG24AM190
Prepared by	MITHUN S
Date Created	5.10.2026

1. Introduction

This Software Design Document (SWD) provides the detailed, implementation-level design of the system described at a high level in the Architecture Document: module decomposition, class/data design (UML), interaction (sequence) design, the database schema, and the API/internal-interface definitions required for the team to divide and implement the work. It maps directly onto the functional requirements (REQ-xx) in the SRS.

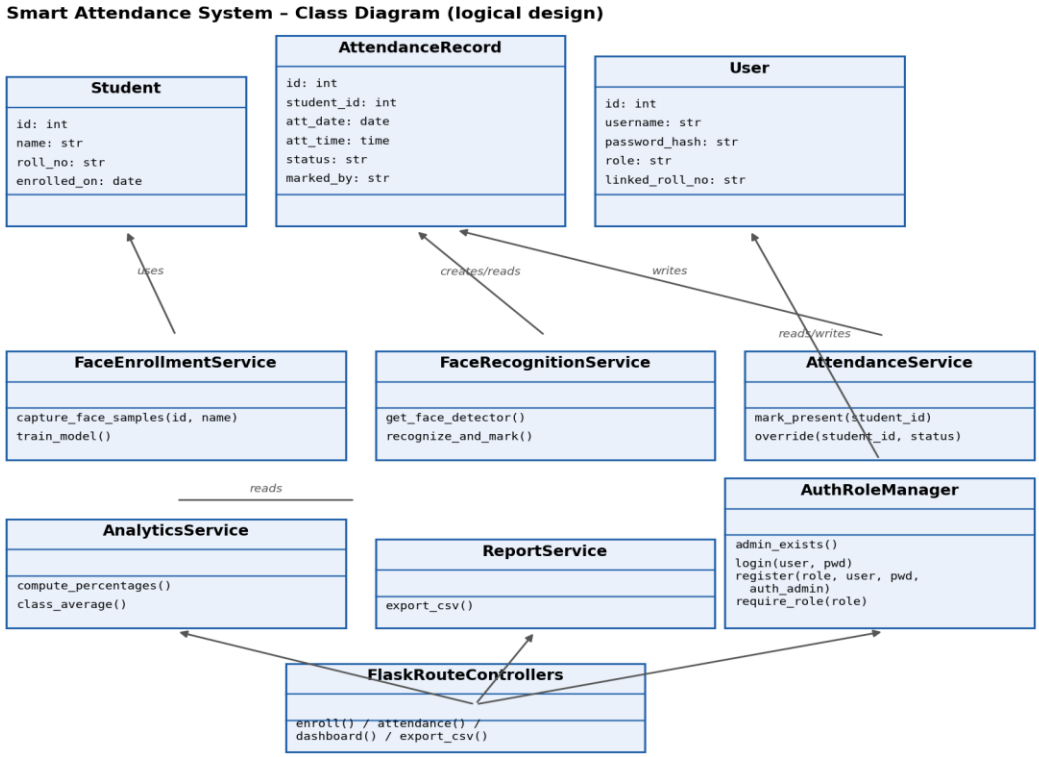
2. Module Decomposition

Module (file)	Responsibility	Owning feature (SRS)
app.py – route controllers	HTTP routing, request validation, session/flash messaging	5.6 Auth
capture_face_samples()	Webcam capture + Haar cascade face detection for enrolment	5.1 Enrolment
train_model()	Rebuilds the LBPH model from all images in dataset/	5.1 Enrolment
recognize_and_mark()	Live recognition + attendance insertion	5.2 Attendance capture
dashboard() / export_csv()	Attendance % computation and CSV export	5.4 Analytics, 5.5 Reports
get_db() / init_db()	SQLite connection and schema setup	all
setup() / admin_exists()	First-run bootstrap: creates the first Admin account, then locks itself	5.6 Auth
register()	Creates new Admin/Faculty accounts, authorized by an existing Admin's credentials	5.6 Auth
login() / logout() / start_session()	Authenticates a user and establishes/clears their session	5.6 Auth

This maps onto a suggested 4-person split: (1) Enrollment & face capture, (2) Recognition & attendance marking, (3) Analytics, reporting & CSV export, (4) Auth/roles & UI/templates – matching the SRS feature sections and satisfying the course requirement that each member owns one full functionality.

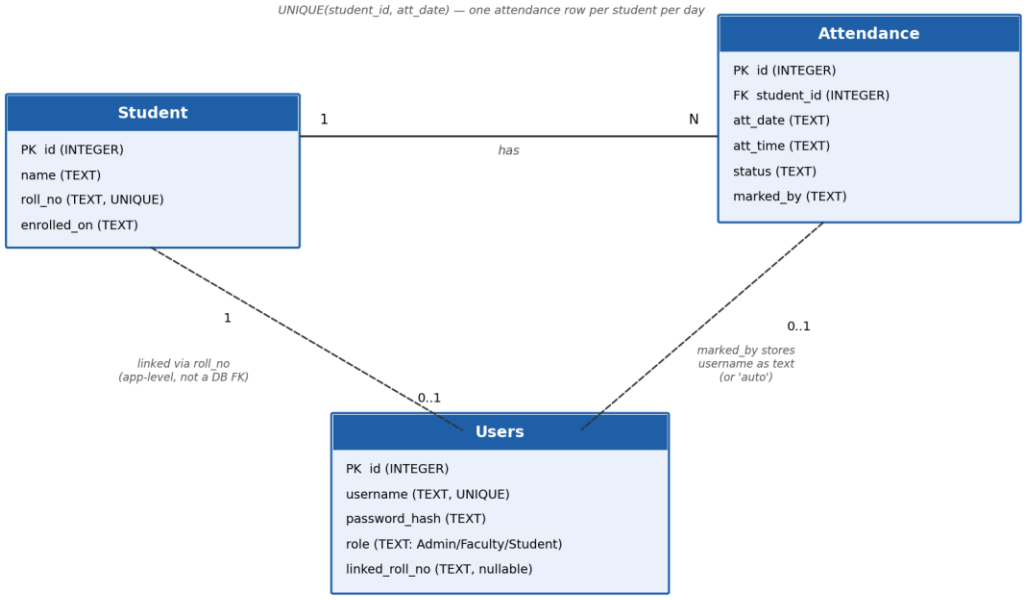
3. Class Diagram (UML)

The diagram below shows the logical design: this mini-project's codebase is currently organised as functions grouped by responsibility rather than formal Python classes, so this is the target object-oriented decomposition the team should refactor toward as the implementation matures (or may implement directly as classes from the start).



Class	Responsibility	Key methods
Student	Data model for an enrolled student	(data only)
AttendanceRecord	Data model for one attendance entry	(data only)
User	Data model for a login account (Admin/Faculty/Student), optionally linked to a Student by roll number	(data only)
FaceEnrollmentService	Captures and stores face samples; retrains the model	capture_face_samples(), train_model()
FaceRecognitionService	Loads the model and recognizes faces live	get_face_detector(), recognize_and_mark()
AttendanceService	Marks/overrides attendance records	mark_present(), override()
AnalyticsService	Computes per-student and class-wide attendance %	compute_percentages(), class_average()
ReportService	Builds the CSV export	export_csv()
AuthRoleManager	First-run admin setup, authorized account creation, login/logout, and role-based access checks	admin_exists(), login(), register(), require_role()
FlaskRouteControllers	Entry point wiring HTTP requests to the above services	enroll(), attendance(), dashboard(), export_csv()

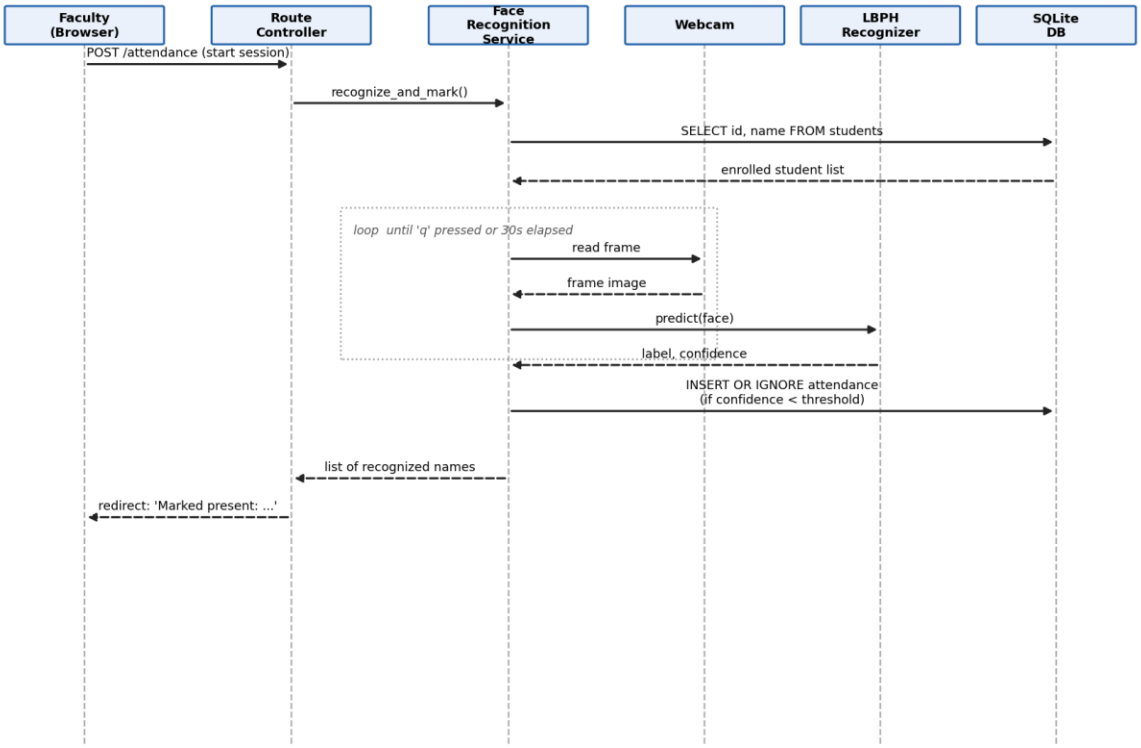
4. Entity-Relationship Diagram



Student (1) has many Attendance records (N); a UNIQUE(student_id, att_date) constraint on Attendance enforces REQ-AT-4/REQ-OV-2's rule of one attendance outcome per student per day (later overrides update, rather than duplicate, that row). This expands on the field layouts already given in SRS Appendix B.

The Users table (added to support REQ-AU-1..4) is linked to Student only loosely, at the application level: a Student-role user's linked_roll_no is matched against Student.roll_no at query time (in the Dashboard and login-enrollment-check logic), not enforced as a database foreign key, since a user account can briefly exist without a matching student record (e.g. right after an Admin/Faculty account is created, or if a student is later un-enrolled). Attendance.marked_by stores a username as plain text (or the literal value 'auto') for the same reason - it is an audit trail, not a referential link.

5. Sequence Diagram: Take Attendance



Steps: (1) Faculty submits the attendance form; (2) the Route Controller calls the Face Recognition Service; (3) the service loads the enrolled student list from SQLite; (4) it loops, reading frames from the webcam and asking the LBPH recognizer to predict a label and confidence for each detected face, until 'q' is pressed or 30 seconds elapse; (5) matches under the confidence threshold are inserted into the attendance table (ignored if already present today); (6) the list of recognized names is returned up to the browser.

5.1 Sequence outline: Enrollment (for comparison)

- Faculty submits the enroll form (name, roll number) -> Route Controller inserts a Student row.
- Route Controller calls FaceEnrollmentService.capture_face_samples(student_id, name).
- Service opens the webcam, loops capturing frames, detects a face per frame via the Haar cascade, and saves cropped grayscale images until 30 samples are captured or the user presses 'q'.
- Route Controller calls FaceEnrollmentService.train_model(), which re-reads every image in dataset/ and retrains the LBPH recognizer, saving trainer/trainer.yml.
- A success/failure flash message is returned to the browser.

5.2 Sequence outline: First-run Admin setup

- Any request, by anyone, before an Admin account exists -> a before_request hook (force_first_admin_setup) intercepts it and redirects to /setup.
- The first visitor chooses any username/password on the /setup form -> AuthRoleManager validates length and confirmation match, then inserts the first Users row with role='Admin'.
- Once that Admin row exists, /setup immediately redirects to /login for every subsequent visit - it cannot be used to create a second Admin this way.

5.3 Sequence outline: Authorized Admin/Faculty registration

- An existing Admin/Faculty's username+password is submitted on /register alongside the new account's desired role/username/password (the 'Create new user' link from the login page).
- AuthRoleManager looks up the submitted auth_username, confirms it belongs to an Admin, and checks the password hash -> wrong credentials reject the whole request before any new account is touched.
- If authorized, the new username is checked against both existing Users and existing Student roll numbers (to avoid an Admin/Faculty username silently colliding with a student's login) before the new Users row is inserted.

6. API Definitions (Flask routes)

Method & Route	Request	Response	Notes
GET/POST /setup	form: username, password, confirm	Creates first Admin; redirects to /login once one exists	First-run only; REQ-AU-1
GET/POST /register	form: role, username, password, confirm, auth_username, auth_password	Creates an Admin/Faculty account if authorization is valid	REQ-AU-1, REQ-AU-4
GET/POST /login	form: username, password	Starts a session; rejects with flash on bad credentials or an unenrolled student	REQ-AU-1, REQ-AU-2
GET /logout	none	Clears the session, redirects to /login	REQ-AU-1
GET /	none	HTML: overview counts (students, sessions, present today)	Home dashboard; login required
GET /enroll	none	HTML: enroll form + roster table	Admin/Faculty only
POST /enroll	form: name, roll_no	Redirect + flash message	Triggers webcam capture + retrain; REQ-EN-1..4; Admin/Faculty only
GET /attendance	none	HTML: capture button + today's present list	Admin/Faculty only
POST /attendance	none (session-triggered)	Redirect to the review screen with the camera's recognized result	Triggers live recognition; REQ-AT-1..5; Admin/Faculty only
GET /attendance/review	none	HTML: editable checklist of the pending recognition result	REQ-OV-1; Admin/Faculty only
POST /attendance/save	form: present_<id> checkboxes	Upserts attendance rows, redirect + flash	REQ-OV-1..3; Admin/Faculty only
GET /dashboard	none	HTML: per-student attendance % table (own record only if Student)	REQ-AN-1..4; login required
GET /export	none	CSV file download (Content-Disposition: attachment)	REQ-RP-1..3; Admin/Faculty only

All routes are server-rendered (Flask + Jinja templates), not a separate JSON REST API, since the UI and backend are not decoupled in this version. Future work could expose these as JSON endpoints for a mobile client.

7. Internal Module Interfaces ('ABI'-equivalent)

Because this system is a single compiled-together Python process rather than separately compiled binaries, there is no true binary interface (ABI) in the C/C++ sense. The equivalent contract for this project is the set of internal Python function signatures each module exposes to the rest of the app – these are the boundaries other modules are allowed to call across, and the ones that must stay stable if a module's internals are later swapped out (e.g. replacing LBPH with a different recognizer).

Function signature	Returns	Raises
capture_face_samples(student_id: int, name: str) -> int	number of face samples captured	RuntimeError if webcam unavailable
train_model() -> bool	True if at least one face was trained on	—
get_face_detector() -> cv2.CascadeClassifier	a loaded Haar cascade detector	RuntimeError if the cascade file is missing/empty
recognize_and_mark() -> list[str]	names of students marked present this session	RuntimeError if no trained model exists, or webcam unavailable
get_db() -> sqlite3.Connection	a row-factory-enabled DB connection	—
init_db() -> None	none (side effect: creates tables if absent)	—
admin_exists() -> bool	True if at least one Admin-role user exists	—
start_session(user: sqlite3.Row) -> None	none (side effect: populates the Flask session)	—

8. Design Patterns Used

- Layered architecture / separation of concerns: route controllers never touch OpenCV or SQL directly in the target design – they delegate to service functions (Section 2), matching the Architecture Document's 3-layer model.
- Service-oriented functions: each feature (enrollment, recognition, analytics) is exposed as a small set of functions with a clear input/output contract (Section 7), similar in spirit to a service layer in MVC.
- Fail-safe default: recognize_and_mark() defaults an unmatched face to 'Unknown' rather than guessing the nearest enrolled student with low confidence, reducing false attendance (tunable via CONFIDENCE_THRESHOLD).

9. Error Handling Strategy

Scenario	Handling
Webcam not available / already in use	RuntimeError is raised, caught by the Flask route, and shown as a red flash message rather than crashing the app.
No face detected during enrollment capture	Frame is skipped silently; if zero samples are captured by the time the user stops, a 'try again' flash message is shown.
Duplicate roll number on enroll	sqlite3.IntegrityError is caught and converted to a user-facing 'already enrolled' flash message.
Attendance taken before any student is enrolled / before a model is trained	RuntimeError with a clear message ('enroll at least one student first'), caught and flashed.

Scenario	Handling
Cascade/model file missing or corrupted	get_face_detector() checks .empty() explicitly and raises a descriptive RuntimeError instead of letting OpenCV's low-level assertion crash the process.
Wrong admin authorization submitted on /register	The whole request is rejected with a flash message before any new account is created; no partial account is ever inserted.
Someone revisits /setup after an Admin already exists	Immediately redirected to /login; the setup form never re-renders once the system is bootstrapped.
A student's account still exists after they are un-enrolled	login() re-checks enrollment status against the students table on every login attempt and rejects access with a clear message, even though the Users row itself was not deleted.
New roll number collides with an existing Admin/Faculty username	enroll() checks for this explicitly and rejects with a flash message before any student row is created.

10. Traceability to SRS Requirements

Requirement	Design element
REQ-EN-1..4	FaceEnrollmentService (capture_face_samples, train_model) – Section 2, 7
REQ-AT-1..5	FaceRecognitionService (recognize_and_mark) – Section 2, 5, 7
REQ-OV-1..3	AttendanceService override path + UNIQUE constraint – Section 4, 9
REQ-AN-1..4	AnalyticsService (dashboard route) – Section 2, 6
REQ-RP-1..3	ReportService (export_csv route) – Section 2, 6
REQ-AU-1..4	AuthRoleManager: admin_exists()/setup(), register(), login()/logout(), require_role() – Section 2, 5.2, 5.3, 6, 7